OSSP (Operating Systems And Systems Programming) - 25CS2104E

ROLL NO: 2520030215
NAME: K. SHASHANK REDDY
SECTION: 08

PRACTICAL - 3

Aim: To Develop a C program using fork() that creates a parent and child process. Display the
Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.
Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated)
using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

void show_status(const char *stage)
{
    char path[100];
    char command[150];

    printf("\n[%s]\n", stage);
    printf("PID  : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    sprintf(path, "/proc/%d/status", getpid());

    sprintf(command, "grep '^State:' %s", path);
    system(command);
}

int main()
{
    pid_t pid;

    printf("=== PROCESS STATE DEMONSTRATION ===\n");
```

```c
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        show_status("Child - Running");

        printf("\nChild PID  : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("\nChild is now WAITING/SLEEPING for 10 seconds...\n");
        sleep(10);

        show_status("Child - Running after sleep");

        printf("\nChild will terminate now.\n");
        exit(0);
    }
    else
    {
        // Parent process
```

```c
        printf("\nChild will terminate now.\n");
        exit(0);
    }
    else
    {
        // Parent process
        show_status("Parent - Running");

        printf("\nParent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        printf("\nParent is WAITING for child using waitpid()...\n");

        waitpid(pid, NULL, 0);

        show_status("Parent - Running after child termination");

        printf("\nChild has terminated.\n");
        printf("Parent process is terminating.\n");
    }

    return 0;
}
```

OUTPUT:

```
Parent PID : 1276
Child PID  : 1279

Parent is WAITING for child using waitpid()...

[Child - Running]
PID  : 1279
PPID : 1276
State:  S (sleeping)

Child PID  : 1279
Child PPID : 1276

Child is now WAITING/SLEEPING for 10 seconds...

[Child - Running after sleep]
PID  : 1279
PPID : 1276
State:  S (sleeping)

Child will terminate now.

[Parent - Running after child termination]
PID  : 1276
PPID : 1216
State:  S (sleeping)

Child has terminated.
Parent process is terminating.
```